

Amazon ML Challenge 2026: Business Entity Resolution

Team Ctrl Alt Del, Indian Institute of Information Technology, Nagpur
Abdul Malik, Rakshit Ladda, Samarth Bhandegaonkar, Ashwin Rathore

Final public leaderboard score: **0.987462** (macro F0.5). Candidate set: **5.14 candidates per Source 1 entity**.

1. How we read the problem

For each Source 1 business we had to find every Source 2 and Source 3 record that describes the same business. Before building anything we spent time just reading records side by side, and a few things shaped every decision after that.

Every S2/S3 record belongs to at most one S1 business. In the training ground truth, all 7.64M matched IDs are unique. So the real question for each S2/S3 record is "which S1 is this, if any?", not "is this pair a match?". We built the final assignment step around that.

The noise on true copies is heavy but mostly harmless. Typos, garbled numbers (11345 became 91345), filler words added to names ("Partners", "Center", "Services"), address parts in a different order, invented replacement names, empty addresses, and names written in Devanagari, Bengali or Kannada script.

The decoys are the hard part. About 26% of training S2/S3 records match nothing. When we looked at them, they were near-copies of a real business with exactly one *real* change: the house number moved by one (159 became 160), a real word was added ("Aligarh Leasing **Group**", "Cultural Fund **Southside**"), or the legal form changed (Pvt Ltd became Limited). A plain string-similarity model scores these almost as high as true copies, which is fatal under a metric that punishes false merges twice as hard as misses.

The test set is not distributed like train. Three differences mattered:

- France appears only in test (259,452 S1 entities, about 15%). An adversarial classifier separates French records from training records almost perfectly (AUC 0.9995, against 0.69 for US and India). The addresses use départements where train would use a region (Nord rather than Hauts-de-France), there are French filler words, and postcodes are usually missing.
- Test has about 5.75 S2/S3 records per S1, against 4.68 in train.
- About 40% of test S2/S3 records look like decoys, roughly 1.9 times as many per S1 as in train.

The metric is macro F0.5 per S1 business, and a business with no true match only scores 1.0 if we predict nothing for it. In practice that means every wrong link costs a whole entity's score, so we tuned everything for precision first.

2. Candidate generation (blocking)

Comparing every record with every other record inside a country would mean about 6.7×10^{12} pairs, so blocking decides both the speed and the recall ceiling of the whole system.

Normalisation. Names and addresses are transliterated to ASCII (unicode), lower-cased and stripped of punctuation. Common abbreviations are expanded (Rd to road, St to street, Pvt to private) and the legal form is split off into its own field, so "Sharma Traders Pvt Ltd" gives a core name "sharma traders" and a legal form "private limited".

Dense retrieval. We fine-tuned `intfloat/multilingual-e5-small` (MIT licence, 118M parameters) as a bi-encoder with `MultipleNegativesRankingLoss`, batch size 512, using same-country records as in-batch negatives. It is trained only on "fold A" of the S1 entities (a hash split; section 5 explains why). Each S2/S3 record takes its 10 nearest S1 records within the same country, using exact GPU search.

Sparse retrieval. TF-IDF over character 3-grams on the name, the address and name+address, with very common 3-grams dropped (`max_df = 0.01`). The top 3 per field are added. This catches records the dense model misses because of heavy typos.

Stage-1 filter. A LightGBM model (127 leaves, 300 rounds) on about 40 cheap pair features scores every retrieved pair, and each S2/S3 record keeps its top 3 S1 candidates.

Final pruning. Only candidates with a stage-1 probability of at least 0.005 are passed to the final model. This took us from 17.3 to **5.14 candidates per S1 entity** (8,906,044 pairs on test) with no loss in validation score. `candidate_pairs.tsv` is exactly this set: the pairs the final model runs inference on.

Recall, measured on held-out S1 entities that the retrieval models never saw:

	Recall
Bi-encoder @1 / @5 / @10 / @20	97.4% / 98.8% / 99.1% / 99.3%
TF-IDF alone	93.8%
Bi-encoder U TF-IDF	99.2%
After stage-1 top-3 and the final filter	98.97%

How big the candidate set is, and how it compares.

Stage	Candidates per S1 entity
No blocking (every S2/S3 record in the same country)	about 3.9 million
Bi-encoder top 10 per S2/S3 record	about 58
Top 3 per record after stage 1	17.3
Final set (stage-1 $p \geq 0.005$), what the final model scores	5.14

Across S1 entities the final set has a median of 5, a 90th percentile of 8 and a 99th percentile of 11. The largest is 164, for a name shared by many records, and 13,305 S1 entities (0.8%) have no candidates at all.

For scale: test has about 5.75 S2/S3 records per S1 in total, decoys included, and we predict about 3.4 matches per S1. So the final model looks at fewer candidates per S1 than there are records per S1, only about 1.5 times the number of links it actually makes, and it still keeps 98.97% of true links on validation. Put another way, each S2/S3 record is attached to fewer than one S1 candidate on average (8.9M pairs for 10.0M records). We also tried going smaller: a stricter filter at $p \geq 0.01$ gave 4.81 candidates per S1 and cost 0.0004 on the leaderboard. So 5.14 is close to the smallest set that doesn't start throwing away true matches on this data.

If the matching model made perfect decisions on this candidate set, validation F0.5 would be 0.9968. Our model reaches 0.9914, so most of what we lose comes from matching decisions, not from blocking. We also tried a much larger BGE-M3 (568M) bi-encoder, and it gave the same recall (99.07% against 99.09%), so we kept the small one.

3. The matching model

3.1 Features

- **Name:** fuzzy ratio, token-set, token-sort and partial ratio, plus Jaro-Winkler, on both the normalised core name and the raw name; legal-form agreement flags; bi-encoder cosine and rank.
- **Extra-word weights:** for words that appear in only one of the two names, a log-odds weight learned on the training folds for how often that word is harmless filler ("Partners", "Center") versus a real difference ("Southside", "Midtown"). This was one of the more useful hand-made features against decoys.
- **Address:** token-set, ratio and partial similarity; Jaccard over digit runs; house-number relations (edit distance, numeric difference, an off-by-one flag, whether the first or any number agrees); empty-address flags; TF-IDF cosines.
- **Query context:** the stage-1 probability, its gap to the best candidate of the same S2/S3 record, its rank, and the number of candidates.
- **Cross-encoder scores** (next section), with their per-record gap and rank.

We deliberately **removed** "competitor count" features such as "how many records point at this S1". They depend on how dense the corpus is, and test is denser than train, so their meaning shifts. Removing them improved validation by 0.0009 and the leaderboard by 0.0006. It was an early hint that consistency between train and test mattered more than adding features.

3.2 Cross-encoders

Hand features can say that two addresses differ by one digit, but they struggle to say whether that digit matters. Models that read both records together were our biggest single improvement: adding our first cross-encoder to v1 moved the leaderboard from 0.956 to 0.982.

- **multilingual-e5-small and multilingual-e5-base** (both MIT) as pair classifiers, trained for one epoch on 1.2M to 1.5M fold-A queries. Their training pairs come from the bi-encoder's own top 3 per record, so they see the true match next to the hard negatives and decoys that retrieval actually produces.
- **France self-training.** France has no labels, so we took our own confident test predictions ($p \geq 0.9$ as positives, $p \leq 0.05$ as negatives) and fine-tuned both cross-encoders on them. This uses only test inputs, never labels. The first round clearly helped (leaderboard 0.9818 to 0.9839); a second round added little.
- **Qwen2.5-1.5B with LoRA** (Apache-2.0). Used as a sequence classifier with the pair written as one prompt ("Record A: ... Record B: ... Same business?"). LoRA rank 16, alpha 32, on all attention and MLP projections, which is 18.5M trainable parameters. The base stays in bf16 and the adapters are trained in fp32, and gradient checkpointing lets it fit a 24 GB GPU slice. It is trained on about 300k pairs (fold-A queries plus 60k France pseudo-labels) and only scores the uncertain pairs, those where the best stage-2 probability is between 0.05 and 0.95.
- **Averaging four Qwen runs.** We trained the same Qwen recipe four times on different samples of training queries (seeds 0, 101, 102, 103) and average the four scores into one feature. The runs agree strongly (correlation about 0.975), but averaging removes run-to-run noise, and it gave +0.0002 on the leaderboard where bigger models had given nothing.

3.3 Stage 2 and the final decision

The final model is LightGBM (63 leaves, 500 rounds) over the stage-1 features, the hand features and the cross-encoder scores. It is trained only on held-out S1 entities that none of the neural models saw during training.

The assignment follows the structure of the problem. Each S2/S3 record goes to its single most probable S1 candidate if that probability is at least the threshold, and otherwise to nobody. One S1 can receive many records. The threshold is **0.5** for US and India, tuned on validation. For France it is **0.7**; section 6 explains how we set it.

4. How the score improved

Version	What changed	Validation	Public LB
v1	Bi-encoder + TF-IDF blocking, LightGBM	0.967	0.956
v2	+ e5-small cross-encoder	0.9886	0.9818
v3a	+ France self-training	0.9888	0.9839
v3b	+ e5-base cross-encoder	0.9899	0.9852
v3c	– density-dependent count features	0.9909	0.9858
v3q	+ Qwen2.5-1.5B LoRA cross-encoder	0.9912	0.98685
pruned	+ candidate pruning (17.3 → 5.14 per S1)	0.9913	0.986975
gens	+ four Qwen runs averaged	0.9914	0.987183
final	+ France-only threshold 0.7	0.9914	0.987462

The gap between validation and the leaderboard (about 0.004 at the end) is the most important number in this table, and we come back to it in section 6.

5. How we validated

The same model outputs feed several stages, so leakage between them was a real risk. The split we settled on:

- S1 entities are split by a hash of their ID. All neural models (bi-encoder, cross-encoders, Qwen) train only on **fold A**.
- A held-out set **E** of fold-B S1 entities is never used to train any neural model. The stage-2 model is trained with out-of-fold predictions and scored on E.
- E is split again into a **tune** half, used for thresholds and every design choice, and a **lockbox** half that we only reported and never tuned on.

After a few leaderboard surprises we also agreed on a rule: a local gain of 0.0002 or less on the lockbox counts as noise, and doesn't justify spending a leaderboard upload.

6. What limited us, and what we learned

Where the remaining errors are. On validation, 62% of our false merges involve a record with **no address** whose name is shared by several S1 businesses ("Optimal Ventures Co." with no address, for example). The same kind of record makes up 69% of the misses where the right candidate was present, and 91% of the misses that blocking never retrieved. Looking at one pair at a time, there is often simply not enough information to tell those candidates apart.

Bigger models stopped helping. A Qwen2.5-7B LoRA scored the same as the 1.5B locally and slightly worse on the leaderboard. Retraining the cross-encoders on more data gained +0.0004 on the lockbox and lost on the leaderboard. On these ambiguous records, more capacity didn't add information.

The bigger loss was France, and our validation could not see it. Validation stayed about 0.004 above the leaderboard to the end. Our validation set only has US and India, so it could never measure France. Two things point at France as the main source of that gap:

- On test, our model is uncertain (probability between 0.1 and 0.9) on 3.7–4.0% of French records, against 1.6–2.0% for US and India.
- We tested this directly on the leaderboard by changing **only** the French threshold and keeping the predictions fixed. Moving it from 0.5 to 0.6 and then 0.7 raised the score from 0.987183 to 0.987378 and then 0.987462, with the gain shrinking at each step. US and India were untouched, so the whole improvement came from France. The model was over-confident on French pairs, and asking for more certainty there paid off.

As a rough estimate: if US and India score on test about what they score on validation (around 0.991), France would have to be around 0.963 to produce our leaderboard score. That is an upper bound, since the higher decoy density in test also costs US and India something.

What did not work, all measured on the lockbox and, where we had uploads, on the leaderboard:

Experiment	Lockbox	Public LB	Outcome
Cross-encoders retrained on 85% of data + Qwen + Qwen 7B	0.9915	0.98678	worse on LB
Qwen 1.5B + Qwen2.5-7B LoRA	0.9913	0.98665	worse on LB
France-only "safe feature" stage 2	—	0.98566	worse on LB
Extra e5 cross-encoders on uncertain records only	0.9901	0.98524	no gain
Harder pruning ($p \geq 0.01$, 4.81 candidates per S1)	0.9912	0.98655	worse on LB
BGE-M3 568M bi-encoder	recall 99.07% vs 99.09%	—	no gain
LightGBM + XGBoost + CatBoost blend	0.9908	—	no gain

Experiment	Lockbox	Public LB	Outcome
Sibling-consistency, name-frequency and house-number/joined-name features	$\leq +0.0001$	—	noise
Separate thresholds for no-address and common-name records (on validation)	$+0.00003$	—	noise
Third round of France self-training (e5-small)	0.99127	—	99.7% of French links unchanged; not used

The main lesson. The changes that transferred to the leaderboard were the ones that kept train and test consistent (removing density-dependent features) or only reduced variance (averaging runs). Changes that looked good locally by adding capacity usually didn't transfer, because the local validation couldn't see the distribution shift.

The France finding only became clear from leaderboard probes on the last day, which is why the fixes below came too late for the challenge.

7. Future scope

None of the following was built during the challenge. These are the directions we would take next, most promising first.

1. **A French-aware normaliser.** France is our biggest remaining loss, and our normalisation was written with US and Indian records in mind. Next we would handle French conventions before blocking and training: map départements to their region (a mapping that can be learned from the provided records themselves), expand street abbreviations such as "av.", "bd", "r." and "pl.", handle "N°" and "bis/ter" house numbers, and strip French filler words. Because this changes the text that every model sees, it means re-running the whole pipeline, which is why it didn't fit in the challenge window.
 2. **France in validation.** Our validation never contained French data, so it could not warn us about the largest source of error. A validation slice built from confident French pseudo-labels, or from French-style noise applied to held-out training pairs, would make France visible when making decisions instead of only through leaderboard probes.
 3. **Per-country calibration instead of threshold probes.** We found France's threshold by probing the leaderboard. A cleaner way is to calibrate the model's probabilities per country, for example with temperature scaling fitted on French pseudo-labels, so that 0.5 means the same thing in every country.
 4. **Deciding ambiguous records jointly.** Most remaining errors are no-address records whose name is shared by several S1 businesses. Scoring one pair at a time can't separate them. A listwise model that sees all candidates of a record at once, or a check that an S2 record and an S3 record that match each other end up at the same S1, could use information that pairwise scoring ignores.
 5. **An adaptive candidate budget.** Instead of one probability cut-off, the number of candidates kept could depend on how clear the decision is. When the best candidate is far ahead, one candidate is enough; only close calls need more. This could cut the candidate set further without losing recall.
 6. **A faster pipeline.** The four-run Qwen ensemble is the slowest part. Distilling it into a single small cross-encoder would cut the end-to-end runtime a lot, which matters if this runs over billions of records.
-

8. Other details

- **Models and licences:** multilingual-e5-small (MIT, 118M), multilingual-e5-base (MIT, 278M), Qwen2.5-1.5B (Apache-2.0, 1.54B), LightGBM (MIT). All are under the 8B limit.
- **Data:** only the provided training and test files. No external data, APIs, geocoding or lookups. The French pseudo-labels come from our own predictions on the test inputs.
- **Hardware:** NVIDIA RTX PRO 6000 Blackwell, split into 24 GB MIG slices; 24 CPU cores and 377 GB RAM. We sharded GPU scoring across up to 8 slices. On a single 24 GB GPU the full pipeline takes roughly 40 to 50 hours.
- **Reproducibility:** `run_final.sh` in the submission package runs every step in order, from raw data to both output files. Seeds are fixed and LightGBM runs deterministically. GPU training can still differ in the last digits between machines, so the `output/` folder holds the exact files we uploaded.